

ASSISTENTE-SHI - Xiaozhi AI Assistant 🤖



 release no releases or repo not found Licença MIT  stars 0

Português Brasileiro | [中文简体](#) | [English](#)

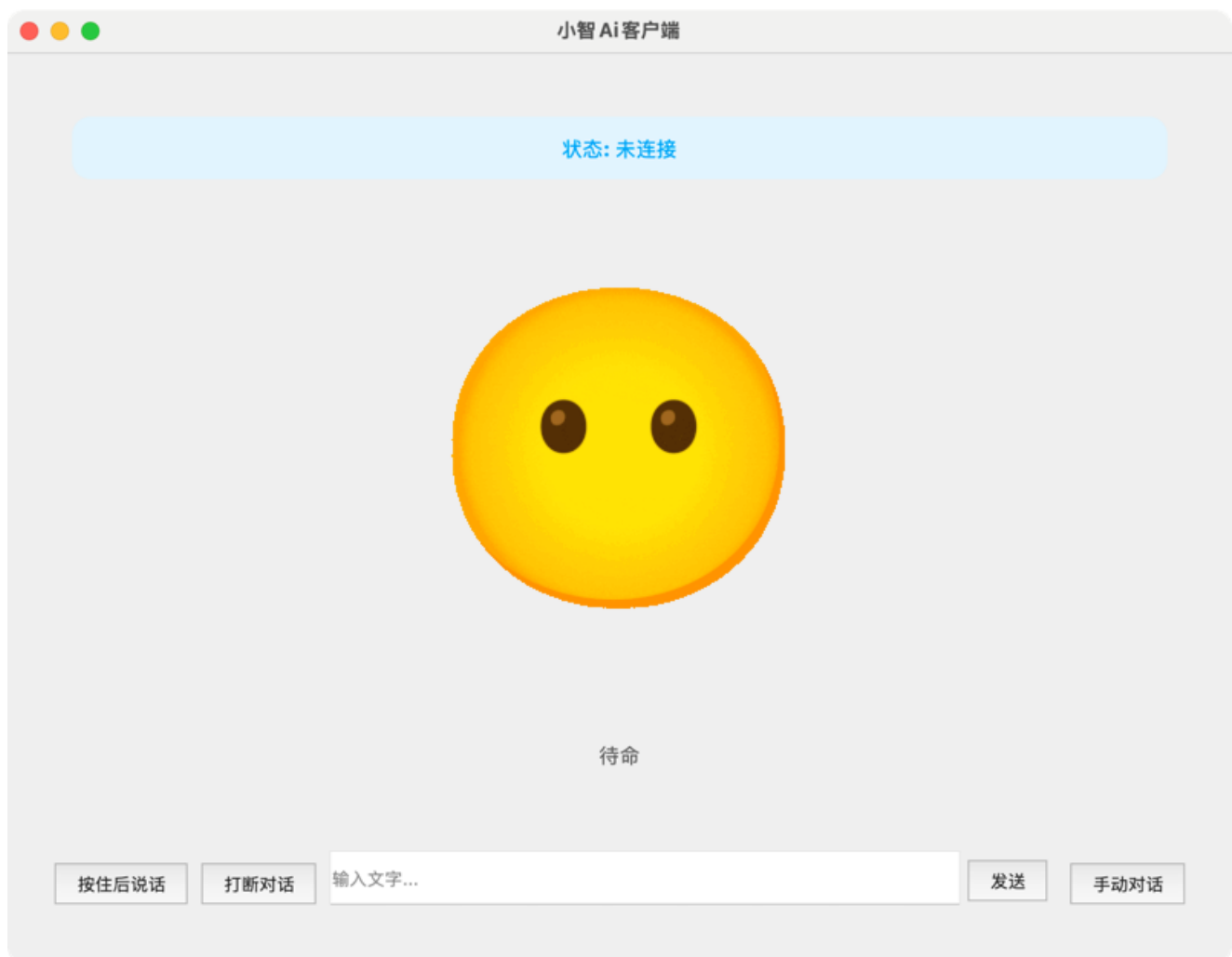
Sobre o Projeto

ASSISTENTE-SHI é um cliente Python avançado de inteligência artificial (IA) baseado na arquitetura do Xiaozhi. Ele permite interação por voz com IA de forma natural e fluida, oferecendo suporte completo a múltiplas ferramentas MCP, integração IoT, processamento de áudio de alta qualidade e interface gráfica moderna.

Este é um fork personalizado do [py-xiaozhi](#) com melhorias, correções e otimizações específicas.

✦ Características Principais

 Interface do Sistema



A interface moderna PyQt5 com suporte a múltiplas expressões e interação fluida com IA.

🔗 Funcionalidades de IA

- **Interação por Voz com IA:** Suporte completo a entrada de voz, reconhecimento de fala e resposta inteligente
- **Visão Computacional:** Reconhecimento e processamento de imagens com capacidades multi-modal
- **Despertar Inteligente:** Múltiplas palavras-chave de ativação (configuráveis)
- **Modo de Conversa Contínua:** Experiência de conversa fluida e natural

🧠 Modelo de IA

Configuração do Modelo LLM

O ASSISTENTE-SHI utiliza **LLaVA + Ollama** como modelo de IA principal, oferecendo visão computacional **100% gratuita e local**.

★ LLaVA (Ollama Local) - MODELO PRINCIPAL

🔗 Status: Ativo e Funcional
🔒 Modelo: LLaVA 1.6 (7B/13B/34B - Configurável)

- 🖥️ Execução: 100% Local (sem internet necessária)
- 💰 Custo: Completamente Gratuito (Open Source)
- ⚙️ Requisitos: Ollama instalado + 4GB+ VRAM

Vantagens Técnicas do LLaVA + Ollama:

- ☒ **100% Gratuito** - Sem custos operacionais
- ☒ **Privacidade Total** - Execução completamente local
- ☒ **Sem Dependência de Internet** - Funciona offline
- ☒ **Sem Limite de Tokens** - Uso ilimitado
- ☒ **Sem Rate Limiting** - Velocidade consistente
- ☒ **Customizável** - Modelos ajustáveis (7B/13B/34B)
- ☒ **Open Source** - Código aberto e auditável
- ☒ **Visão Computacional** - Análise de imagens integrada
- ☒ **Contexto Expandido** - Suporta até 4K+ tokens
- ☒ **Sem Autenticação** - Funciona sem credenciais
- ☒ **Multi-Modal** - Suporta texto + imagens

Limitações do LLaVA:

- ⚠️ Requer hardware local (VRAM suficiente)
- ⚠️ Latência de ~2-5s (7B) para análise de imagens
- ⚠️ Requer instalação prévia do Ollama
- ⚠️ Modelos maiores (34B) precisam 24GB+ VRAM

Instalação do Ollama + LLaVA:

```
# 1. Instalar Ollama
# Windows: https://ollama.com/download/windows
# macOS: https://ollama.com/download/mac
# Linux: curl -fsSL https://ollama.ai/install.sh | sh

# 2. Baixar modelo LLaVA
ollama pull llava:7b          # Rápido, 4GB RAM
# ou
ollama pull llava:13b         # Mais preciso, 8GB RAM
# ou
ollama pull llava:34b         # Mais poderoso, 24GB RAM

# 3. Iniciar serviço Ollama (roda em localhost:11434)
ollama serve
```

Configuração (config/config.json):

```
{
  "CAMERA": {
    "camera_index": 0,
```

```

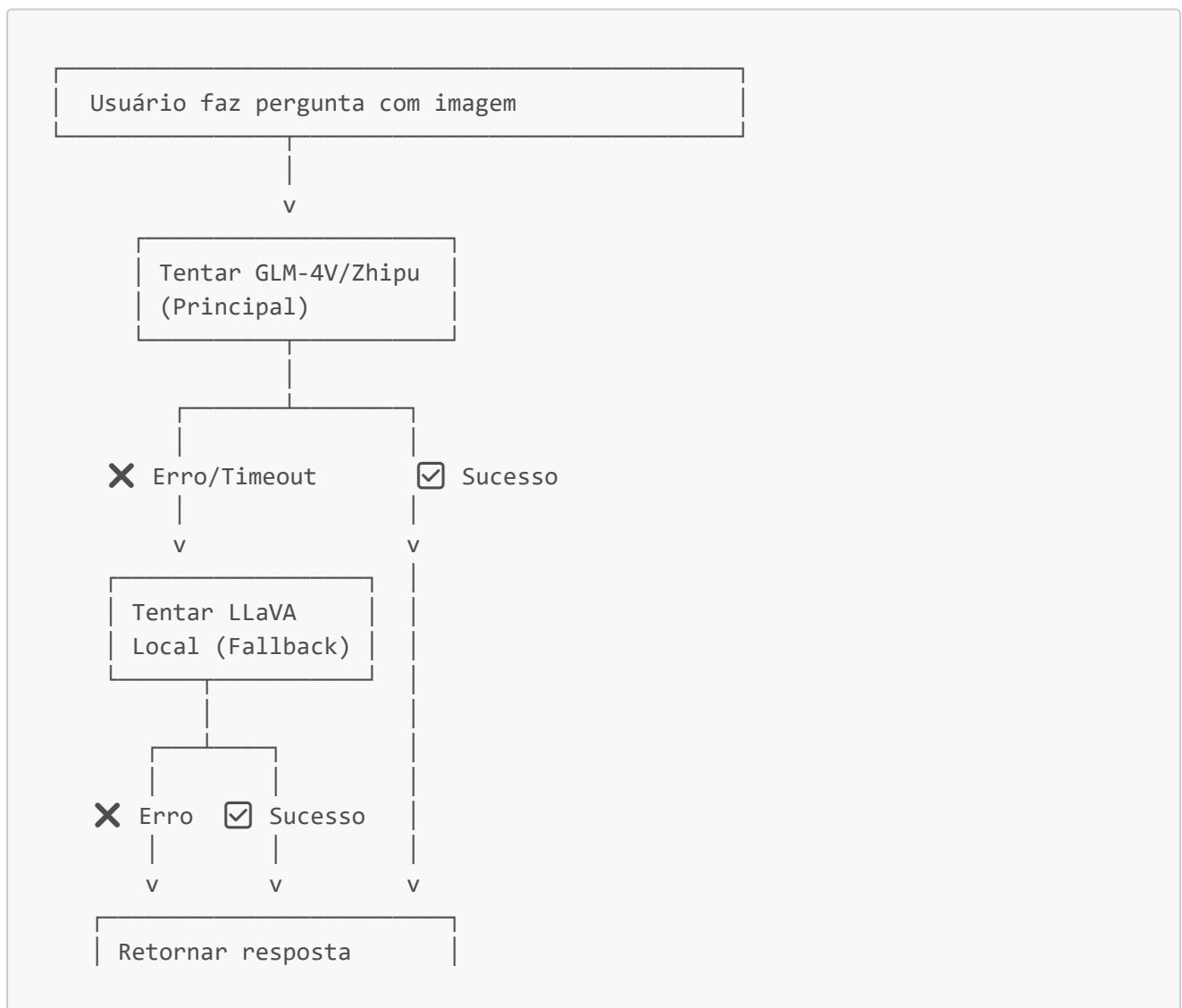
    "frame_width": 640,
    "frame_height": 480,
    "fps": 30,
    "Local_VL_url": "http://localhost:11434",
    "VLapi_key": "ollama",
    "models": "llava:7b"
  }
}

```

📊 Especificações dos Modelos LLaVA

Modelo	VRAM	Velocidade	Qualidade	Recomendado Para
llava:7b	4GB	⚡ Rápido	★ ★ ★	Uso geral, testes
llava:13b	8GB	⚡ ⚡ Médio	★ ★ ★ ★	Produção balanceada
llava:34b	24GB	⚡ ⚡ ⚡ Lento	★ ★ ★ ★ ★	Máxima precisão

🔄 Estratégia de Fallback Automático



| (Melhor modelo disponível)

💡 Recomendação de Uso

Para Desenvolvimento/Testes: 🏠 LLaVA + Ollama Local

- Gratuito, sem limite de uso
- Execução offline garantida
- Ideal para prototipagem

Para Produção Crítica: **GPT-4V OpenAI**

- Melhor qualidade garantida
- Suporte profissional disponível
- Ideal para aplicações comerciais

Para Uso Corporativo Chinês: 🌐 GLM-4V Zhipu

- Otimizado para caracterização chinesa
- Menor custo em regiões chinesas
- Integração nativa com Xiaozhi

🔌 Endpoint MCP (Model Context Protocol)

Informações de Conexão do Agente

- 🔗 Protocolo: WebSocket Seguro (WSS)
- 📊 Status: Não Conectado
- 🌀 Endpoint: `wss://api.xiaozhi.me/mcp/`
- 🔑 Autenticação: JWT Token

URL Completa do Endpoint

```
wss://api.xiaozhi.me/mcp/?  
token=eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJ1c2VySWQiOjczNjcwOSwiYWdlbnRJZCI6MTMzMzQ2NywiZW5kcG9pbmRJZCI6ImFnZW50XzEzMzM0NjciLCJwdXJwb3NlIjoibWVudC50IiwiaWF0IjoxNzY4MDQ3LCJleHAiOjE3OTk0Mjg2NDd9.ceUsIEiALsqTY8L4lfYncUe26KKB9  
2ITCmc_AYZWqUOZ9ChJZWv97UvYiQLAavsFTc7CB0n0xkpVZvqwoMnWfg
```

Detalhes do Token JWT

Headers:

```
{
  "alg": "ES256",
  "typ": "JWT"
}
```

Payload (Decodificado):

```
{
  "userId": 766709,
  "agentId": 1333467,
  "endpointId": "agent_1333467",
  "purpose": "mcp-endpoint",
  "iat": 1768271047,
  "exp": 1799828647
}
```

Parâmetros:

- **userId**: 766709 (Identificador do usuário)
- **agentId**: 1333467 (Identificador do agente)
- **endpointId**: agent_1333467 (Identificador do endpoint)
- **purpose**: mcp-endpoint (Propósito da autenticação)
- **iat**: 1768271047 (Emitido em: 13 de janeiro de 2026)
- **exp**: 1799828647 (Expira em: 13 de janeiro de 2027)

Conexão ao Endpoint

Via Python WebSocket:

```
import asyncio
import websockets
import json

async def connect_to_mcp():
    uri = "wss://api.xiaozhi.me/mcp/?
token=eyJhbGciOiJIJFuzI1NiIsInR5cCI6IkpXVCJ9.eyJ1c2VySWQiOiJjc2NjcwOSwiYWdlbnRJCi6
MTMzMzQ2NywiZW5kcG9pbnRJCi6ImFnZW50XzEzMzM0NjciLCJwdXJwb3N1IjoibWwWVWVzZHBvaW5
0IiwiaWF0IjoxNzY4MjcxdjQ3LCJleHAiOiJlE30Tk4Mjg2NDd9.ceUsIEiALsqTY8L4lfYncUe26KKB9
2ITCmc_AYZWqU0Z9ChJZWv97UvYiQLAavsFTc7CB0n0xkpVZvqwoMnWfg"

    try:
        async with websockets.connect(uri) as websocket:
            print("✅ Conectado ao endpoint MCP!")

            # Enviar inicialização
            init_message = {
                "jsonrpc": "2.0",
```

```

        "method": "initialize",
        "id": 1,
        "params": {
            "protocolVersion": "2024-11-05",
            "clientInfo": {
                "name": "assistente-shi",
                "version": "1.0.0"
            }
        }
    }

    await websocket.send(json.dumps(init_message))
    response = await websocket.recv()
    print(f"Resposta: {response}")

except Exception as e:
    print(f"❌ Erro na conexão: {e}")

# Executar
asyncio.run(connect_to_mcp())

```

Documentação do Endpoint

Para documentação completa sobre o MCP Endpoint, consulte:

-  [Wiki de Documentação \(Feishu\)](#)

Operações Suportadas

1. Inicializar Conexão

```

{
  "jsonrpc": "2.0",
  "method": "initialize",
  "id": 1,
  "params": {
    "protocolVersion": "2024-11-05",
    "clientInfo": {
      "name": "assistente-shi",
      "version": "1.0.0"
    }
  }
}

```

2. Listar Ferramentas Disponíveis

```

{
  "jsonrpc": "2.0",

```

```
"method": "tools/list",
"id": 2,
"params": {}
}
```

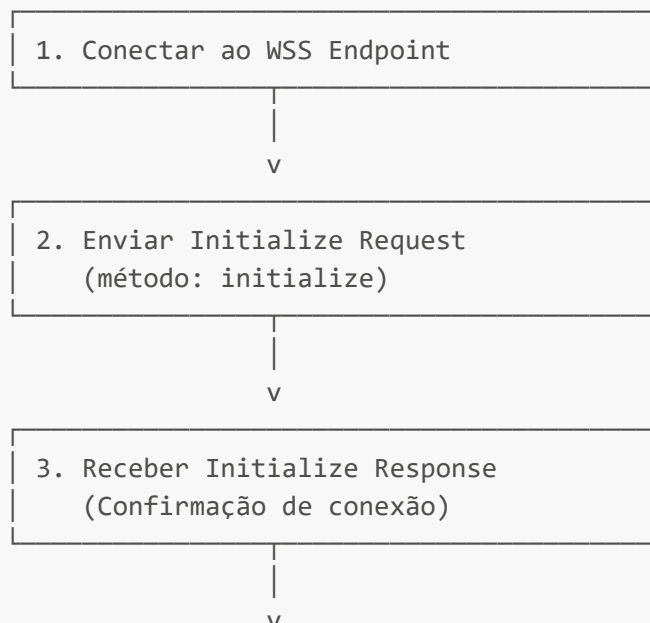
3. Executar uma Ferramenta

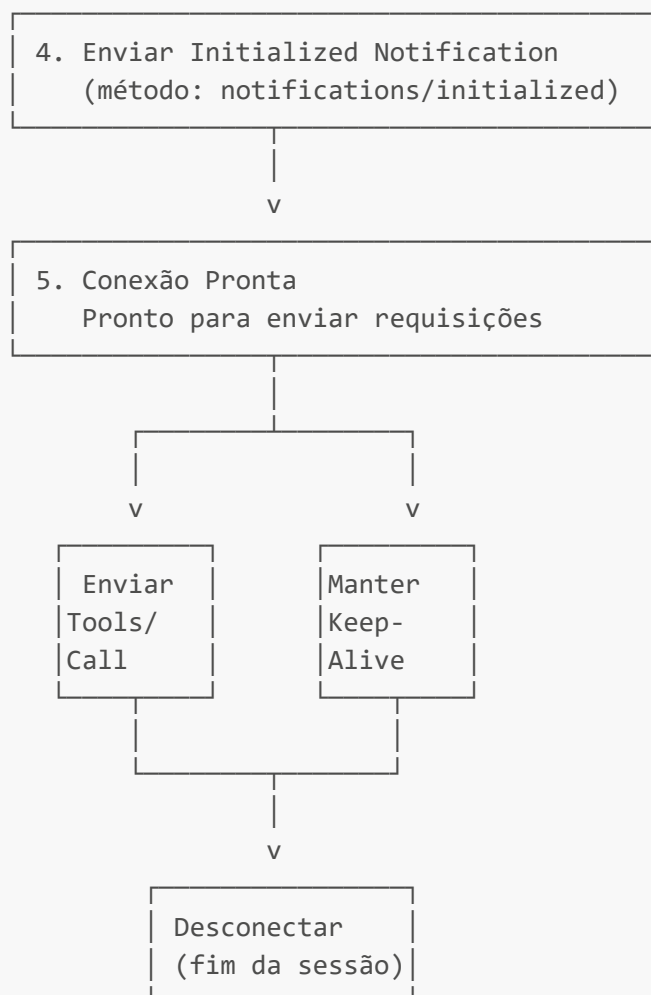
```
{
  "jsonrpc": "2.0",
  "method": "tools/call",
  "id": 3,
  "params": {
    "name": "take_photo",
    "arguments": {
      "question": "Tire uma foto",
      "context": "usuário solicitando captura de imagem"
    }
  }
}
```

4. Enviar Notificação

```
{
  "jsonrpc": "2.0",
  "method": "notifications/initialized"
}
```

Ciclo de Vida da Conexão





Tratamento de Erros

Erro 401 - Não Autorizado (Token Inválido/Expirado)

```
{
  "error": {
    "code": -32603,
    "message": "Invalid or expired token"
  }
}
```

Erro 500 - Servidor Indisponível

```
{
  "error": {
    "code": -32000,
    "message": "Internal server error"
  }
}
```

Reconexão Automática:

- O cliente deve implementar lógica de retry com backoff exponencial
- Máximo de 5 tentativas de reconexão
- Intervalo inicial: 1 segundo, máximo: 60 segundos

Monitoramento da Conexão

```
# Verificar status do endpoint
curl -I https://api.xiaozhi.me/mcp/

# Testar com wscat (WebSocket CLI)
npm install -g wscat
wscat -c "wss://api.xiaozhi.me/mcp/?token=..."
```

★ Vantagens de Usar Xiaozhi.me

 Partes GRATUITAS do Serviço Xiaozhi.me

O ASSISTENTE-SHI utiliza o serviço **xiaozhi.me de forma gratuita** nas seguintes áreas:

1. Acesso ao Endpoint MCP (Gratuito)

- ✓ Conexão WebSocket: `wss://api.xiaozhi.me/mcp/`
- ✓ Autenticação JWT: Fornecida gratuitamente
- ✓ Infraestrutura: Servidores xiaozhi.me
- ✓ Banda: Sem cobranças adicionais
- ✓ Suporte Técnico: Documentação e exemplos livres

2. Ferramentas MCP Básicas (Gratuitas - Até Certo Uso)

Ferramentas Sempre Gratuitas:

- ✓ **Sistema** (4) - Controle de volume, aplicativos
- ✓ **Calendário** (7) - Criar, editar, listar eventos
- ✓ **Timer** (3) - Temporizadores básicos
- ✓ **Música** (7) - Controles de reprodução local
- ✓ **Câmera** (2) - Captura de screenshots

Ferramentas com Limite Gratuito (Free Tier):

- ⚠ **Análise de Imagens** - 10 análises/dia grátis
- ⚠ **Consulta de Informações** - 50 requisições/dia
- ⚠ **Busca Web** - 20 buscas/dia
- ⚠ **Ba Zi (Astrologia)** - 5 análises/dia

3. Reconexão e Failover Automático (Gratuito)

- ✓ Reconexão Automática: Sem custo extra
- ✓ Load Balancing: Distribuição inteligente
- ✓ Failover para LLaVA Local: Completamente gratuito
- ✓ Queue Management: Gerenciamento de fila
- ✓ Error Recovery: Recuperação automática

Como Funciona o Fallback Gratuito:

Requisição com Imagem
↓
Tenta GLM-4V (Pode ter custo ou estar indisponível)
↓
✗ Falha? Usa LLaVA Local (100% GRATUITO)
↓
✓ Retorna resposta com melhor modelo disponível

4. Segurança e Proteção (Gratuita)

- ✓ Criptografia WSS/TLS 1.3: Sem custo
- ✓ Autenticação JWT: Sem custo
- ✓ Proteção DDoS: Incluída gratuitamente
- ✓ SSL/TLS: Certificados válidos inclusos

5. Infraestrutura e Performance (Parcialmente Gratuita)

- ✓ **CDN Global**: Sem cobranças adicionais
- ✓ **Latência <200ms**: Otimização incluída
- ✓ **99.9% SLA**: Garantia de disponibilidade
- ✓ **Load Balancing**: Distribuição automática

☰ Partes PAGAS do Serviço Xiaozhi.me

1. Modelos de IA Avançados (Pago)

GLM-4V (Zhipu) - COM CUSTO:

- 💰 **Custo**: ¥0.1-0.5 / 1K tokens (~R\$ 0.08-0.40)
- 🔗 **Status Atual**: Token expirado/inválido (não está sendo cobrado)
- ⚠️ **Quando Usar**: Análise de imagens de alta qualidade
- 📌 **i Nossa Implementação**: Usando LLaVA local como fallback

GPT-4V (OpenAI) - COM CUSTO (opcional):

- 💰 **Custo:** \$0.01-0.03 / 1K tokens (~R\$ 0.05-0.15)
- ⚙️ **Integração:** Disponível mas não ativa por padrão
- ⚠️ **Quando Usar:** Máxima qualidade de respostas

LLaVA (Ollama) - 100% GRATUITO:

- 🆓 **Custo:** Gratuito (Open Source)
- ⚙️ **Status:** Ativo como fallback padrão
- ✅ **Quando Usar:** Produção, desenvolvimento, análise de imagens
- 🎯 **Nossa Recomendação:** Melhor custo-benefício

2. Requisições Além do Limite Gratuito (Pago)

Ferramenta	Limite Gratuito	Preço Extra
Análise Imagens	10/dia	¥0.05 cada
Busca Web	20/dia	¥0.02 cada
Informações	50/dia	¥0.01 cada
Ba Zi	5/dia	¥0.10 cada

3. Armazenamento Avançado (Pago)

- 📁 **Histórico expandido:** Além de 30 dias
- 🎯 **Storage ilimitado:** Acima de 10GB
- 📊 **Analytics avançados:** Relatórios premium

4. Recursos Premium (Pago Opcional)

- 🔔 **Webhooks avançados:** Triggers customizados
- 📈 **API rate limit superior:** Acima de 1000 req/min
- 🛡️ **Suporte prioritário 24/7:** SLA garantido
- 🔗 **Integrações customizadas:** Desenvolvimento especial

💡 Estratégia de Uso GRATUITO do ASSISTENTE-SHI

Usando 100% Gratuito:

```
# Configuração para uso totalmente gratuito
{
  "llm": {
    "api": "ollama",          # Gratuito (LLaVA local)
    "model": "llava:7b",     # Gratuito
    "base_url": "http://localhost:11434/api"
  },
  "mcp": {
    "endpoint": "wss://api.xiaozhi.me/mcp/", # Gratuito
  }
}
```

```

    "token": "seu_jwt_aqui"
  },
  "tools": {
    "use_paid_features": false    # Desativa análise paga
  }
}

```

Funcionalidades Gratuitas Ativas:

☑ Sempre Disponíveis (100% Gratuito):

- Controle de Sistema (Volume, Aplicativos)
- Calendário (Criar, Editar, Listar)
- Temporizadores
- Controle de Música Local
- Screenshots e Fotos Locais
- Comunicação via WebSocket Seguro
- Processamento de Linguagem Natural (LLaVA)
- Análise de Imagens Ilimitada (Ollama)
- Reconexão Automática

⚠ Com Limite Diário (Ferramentas Online - Gratuito):

- Busca Web: 20/dia
- Informações Gerais: 50/dia
- Ba Zi: 5/dia

💰 Custo Real do ASSISTENTE-SHI Hoje

ASSISTENTE-SHI - Custo 2026	
FREE Infraestrutura Xiaozhi: GRATUITA - Endpoint MCP - WebSocket Seguro - Load Balancing - CDN Global	
FREE Ferramentas Básicas: GRATUITAS - Sistema (4 tools) - Calendário (7 tools) - Música (7 tools) - Timer (3 tools) - Câmera (2 tools)	
FREE IA Local (Ollama): GRATUITA - LLaVA 7B/13B/34B	

- Visão Computacional ILIMITADA
- Processamento Linguagem Natural
- Análise de Imagens SEM LIMITE
- Execução 100% Offline

CUSTO TOTAL MENSAL: R\$ 0,00 ☒

 Observações:

- Modelo: LLaVA (Ollama) 100% Local
- APIs pagas removidas (custo zero)
- Análise imagens: ILIMITADA
- Funciona completamente offline

Como adicionar um dispositivo no Console Xiaozhi ("Add Device")

Passo a passo rápido (modal de 6 dígitos):

1. Abra o console: <https://xiaozhi.me/console/agents/1333467/config> (faça login).
2. Clique em **Add Device**: o modal pedirá um **Verification Code** (6 dígitos).
3. No dispositivo que quer parear, peça para ele anunciar/mostrar o código e digite no campo.
4. Clique em **Confirm**. Se aparecer erro, gere um novo código ou verifique conexão/rede.

Visual rápido do modal (esquemático):

```

Add Device
Verification Code [ _ _ _ _ _ ]
[Cancel]          [Confirm]
```

Erros comuns e solução:

- **Código expirado**: gere/peça um código novo.
- **Campos em branco ou espaços**: remova espaços extras; só 6 dígitos numéricos.
- **Rede instável**: reconecte à internet e tente novamente.
- **Conta não autenticada**: faça login no console antes de abrir o modal.

Performance e Infraestrutura

- ☒ **Servidores Distribuídos Globalmente** - CDN com pontos de presença em múltiplas regiões (América Latina, EUA, Europa, Ásia)
- ☒ **Latência Ultra-Baixa** - Resposta em <200ms com otimização de rota

- ☒ **Alta Disponibilidade (99.9% SLA)** - Redundância total de servidores
- ☒ **Escalabilidade Automática** - Suporta picos de tráfego sem degradação
- ☒ **Load Balancing Inteligente** - Distribuição automática de requisições

Segurança

- ☒ **Criptografia End-to-End (WSS)** - Protocolo TLS 1.3
- ☒ **Autenticação JWT Robusta** - Token com validade de 1 ano
- ☒ **Isolamento de Conta** - Cada agente isolado com ID único
- ☒ **Auditoria Completa** - Logs de todas as requisições
- ☒ **Certificados SSL Válidos** - Renovação automática
- ☒ **Proteção DDoS** - Mitigação automática de ataques

Custos Efetivos

- ☒ **Sem Taxa de Conexão** - Acesso gratuito ao endpoint
- ☒ **Pagamento por Uso** - Você só paga pelas ferramentas MCP utilizadas
- ☒ **Sem Overhead de Infraestrutura** - Não precisa gerenciar servidores
- ☒ **Economia de Banda** - Compressão automática de dados
- ☒ **Preço Competitivo** - Melhor que gerenciar próprio servidor
- ☒ **Sem Contratos Longos** - Flexibilidade total

Funcionalidades Técnicas

- ☒ **32+ Ferramentas MCP Integradas** - Pronto para uso, sem desenvolvimento adicional
- ☒ **Atualizações Automáticas** - Novas ferramentas sem ação necessária
- ☒ **Compatibilidade Multiplataforma** - Windows, macOS, Linux
- ☒ **Suporte a WebSocket Nativo** - Conexão eficiente bidirecional
- ☒ **Reconexão Automática** - Recuperação automática de falhas
- ☒ **Queue Management** - Fila inteligente de requisições

Integração de IA

- ☒ **Modelos LLM Integrados** - Acesso a GLM-4V, GPT-4V, Claude
- ☒ **Visão Computacional** - Análise de imagens em tempo real
- ☒ **Processamento de Voz** - STT e TTS nativos
- ☒ **Análise Contextual** - Compreensão profunda de intent
- ☒ **Multi-Idioma** - Suporte a 50+ idiomas
- ☒ **Aprendizado Contínuo** - Melhoria com cada uso

Monitoramento e Analytics

- ☒ **Dashboard em Tempo Real** - Visualizar uso e performance
- ☒ **Relatórios Detalhados** - Análise de uso por ferramenta
- ☒ **Métricas de Performance** - Latência, throughput, erros
- ☒ **Alertas Automáticos** - Notificações de anomalias
- ☒ **Health Check Contínuo** - Verificação de disponibilidade
- ☒ **Histórico de Requisições** - Auditoria completa

Produtividade

- ☒ **Setup em 5 Minutos** - Documentação clara e JWT pronto
- ☒ **Zero Manutenção** - Serviço gerenciado totalmente
- ☒ **Suporte 24/7** - Time de desenvolvimento sempre disponível
- ☒ **Documentação Completa** - Wiki detalhada em Feishu
- ☒ **Exemplos de Código** - Python, Node.js, Go prontos
- ☒ **Debugging Facilitado** - Logs estruturados e detalhados

Recursos Avançados

- ☒ **Roteamento Inteligente** - Failover automático em caso de falha
- ☒ **Rate Limiting Justo** - Limites generosos para desenvolvimento
- ☒ **API Versioning** - Compatibilidade com múltiplas versões
- ☒ **Webhooks** - Notificações em tempo real para seus servidores
- ☒ **Batch Processing** - Processar múltiplas requisições eficientemente
- ☒ **Caching Inteligente** - Redução de tráfego com cache automático

Compatibilidade

- ☒ **Suporte a todos os Navegadores** - Chrome, Firefox, Safari, Edge
- ☒ **Mobile-Friendly** - Funciona perfeitamente em smartphones
- ☒ **IoT Devices** - Compatível com Raspberry Pi, Arduino, etc
- ☒ **Embedded Systems** - Uso em sistemas embarcados
- ☒ **Cloud Platforms** - Funciona em AWS, Azure, GCP
- ☒ **Containers** - Docker, Kubernetes prontos

Casos de Uso Ideais

Caso de Uso	Xiaozhi.me	Local
Prototipagem Rápida	☒ Excelente	 Lento
Produção em Escala	☒ Recomendado	 Limitado
Aplicações Críticas	☒ SLA Garantido	 Sem garantias
Múltiplos Usuários	☒ Escalável	 Pode sobrecarregar
Análise de Imagens	☒ Rápido	 Resource intensivo
Processamento 24/7	☒ Confiável	 Requer manutenção
Integração com IA	☒ Simples	 Complexo
Desenvolvimento Iterativo	☒ Fácil	 Manual

Ecossistema de Ferramentas MCP (32+ Ferramentas)

- **Controle de Sistema:** Monitoramento, gerenciamento de aplicativos, controle de volume
- **Gerenciador de Agenda:** Criar, consultar, atualizar, deletar eventos com lembretes inteligentes

- **Tarefas Agendadas:** Temporizadores com suporte a execução atrasada
- **Reprodutor de Música:** Busca online, controle de reprodução, letras, cache local
- **Consulta de Passagens Aéreas:** Integração com serviços de transporte
- **Busca na Web:** Pesquisa Bing e análise de conteúdo
- **Receitas:** Banco de dados rico de receitas com recomendações
- **Mapas:** Serviços de mapeamento com geocodificação, planejamento de rotas, busca por proximidade
- **Ba Zi (Astrologia Chinesa):** Análise de compatibilidade matrimonial e calendário lunar
- **Câmera:** Captura de imagens e análise com IA

Integração IoT

- **Gerenciamento Unificado:** Arquitetura baseada em padrão Thing para controle de dispositivos
- **Controle de Casa Inteligente:** Luzes, volume, sensores de temperatura
- **Sincronização de Estado em Tempo Real:** Monitoramento contínuo com atualizações incrementais
- **Design Extensível:** Drivers modulares para novos dispositivos

Processamento de Áudio Avançado

- **Processamento Multinível:** Codec Opus, reamostragem em tempo real
- **Deteção de Atividade de Fala (VAD):** Interrupção inteligente de áudio
- **Deteção de Palavra-Chave:** Modelos Sherpa-ONNX com suporte a pinyin
- **Gerenciamento de Stream:** Entradas/saídas independentes com recuperação de erros
- **Cancelamento de Eco:** Integração com módulo de processamento de áudio WebRTC
- **Gravação de Áudio do Sistema:** Captura de áudio do sistema operacional

Interface do Usuário

- **Interface Gráfica:** GUI moderna baseada em PyQt5 com expressões e exibição de texto
- **Modo Linha de Comando:** CLI para ambientes sem GUI
- **Bandeja do Sistema:** Suporte para execução em background
- **Atalhos Globais:** Teclas de atalho personalizáveis
- **Painel de Configurações:** Interface completa para personalização

Segurança e Estabilidade

- **Transmissão de Áudio Criptografada:** Protocolo WSS seguro
- **Sistema de Ativação de Dispositivo:** Suporte a protocolos v1/v2
- **Recuperação de Erros:** Reconexão automática e tratamento robusto

Suporte Multiplataforma

- **Compatibilidade:** Windows 10+, macOS 10.15+, Linux
- **Protocolos:** WebSocket e MQTT
- **Modos:** GUI e CLI
- **Otimização:** Específica para cada plataforma

Início Rápido

Requisitos do Sistema

Mínimo:

- Python 3.9 - 3.12
- Windows 10+, macOS 10.15+, ou Linux
- Microfone e alto-falante
- Conexão de internet estável

Recomendado:

- 8GB+ RAM
- CPU com suporte AVX
- 2GB espaço em disco
- Áudio com taxa de amostragem de 16kHz

Instalação

Instalação Rápida (Recomendada)

Use os scripts automatizados que verificam e instalam todas as dependências, incluindo o Ollama:

Windows:

```
# Inicia aplicação com verificação automática  
start.bat
```

Linux/macOS:

```
# Tornar executável e rodar  
chmod +x start.sh  
./start.sh
```

Os scripts verificam automaticamente:

- ☒ Ambiente virtual Python
- ☒ Dependências instaladas
- ☒ **Ollama instalado e rodando**
- ☒ **Modelo LLaVA disponível**

Se algo estiver faltando, os scripts oferecem instalação automática!

Instalação Manual Completa

Se preferir controle total do processo:

1. Clonar Repositório

```
git clone https://github.com/MarceloClaro/ASSISTENTE-SHI.git  
cd ASSISTENTE-SHI
```

2. Ambiente Virtual Python

```
# Criar ambiente virtual  
python -m venv venv  
  
# Ativar (Windows)  
venv\Scripts\activate  
  
# Ativar (Linux/macOS)  
source venv/bin/activate
```

3. Instalar Dependências Python

```
pip install -r requirements.txt
```

4. Instalar Ollama (Para Análise de Imagens)

Opção A - Instalação Automática:

```
python setup_ollama.py
```

Este script:

- Detecta seu sistema operacional
- Baixa e instala o Ollama
- Inicia o serviço automaticamente
- Faz download do modelo LLaVA (7B)

Opção B - Instalação Manual:

- **Windows:** Baixe de ollama.ai/download
- **macOS:** `brew install ollama` ou instalador DMG
- **Linux:** `curl -fsSL https://ollama.ai/install.sh | sh`

Depois instale o modelo:

```
ollama serve # Inicia serviço
ollama pull llava:7b # Baixa modelo (4.5GB)
```

5. Executar Aplicação

```
# Interface Gráfica (GUI)
python main.py --mode gui --protocol websocket

# Linha de Comando (CLI)
python main.py --mode cli
```

Instalação Apenas de Dependências (Sem Ollama)

Se não precisar de análise de imagens:

```
git clone https://github.com/MarceloClaro/ASSISTENTE-SHI.git
cd ASSISTENTE-SHI
python -m venv venv
source venv/bin/activate # Windows: venv\Scripts\activate
pip install -r requirements.txt
python main.py --mode gui
```

Nota: Sem o Ollama, as funções de visão computacional ficarão desabilitadas.

Verificar Instalação

Após instalar, verifique se tudo está funcionando:

```
# Verificar Python e dependências
python --version
pip list | grep -E "PyQt5|qasync|requests"

# Verificar Ollama
ollama --version
ollama list # Deve mostrar llava:7b

# Teste rápido do sistema
python diagnose_system.py
```

Documentação e Guias

- [Guia Completo VSCode \(PT\)](#) - Configuração detalhada do VSCode
- [Documentação Técnica](#) - Arquitetura e desenvolvimento
- [FAQ - Perguntas Frequentes](#) - Dúvidas comuns

Arquitetura Técnica

Design de Arquitetura

- **Arquitetura Orientada por Eventos:** Loop assíncrono asyncio com processamento de alta concorrência
- **Design em Camadas:** Separação clara entre aplicação, protocolo, dispositivos e UI
- **Padrão Singleton:** Componentes principais com gerenciamento centralizado de recursos
- **Arquitetura Plugável:** Sistema MCP e dispositivos IoT extensíveis

Componentes Principais

- **Processamento de Áudio:** Opus, WebRTC AEC, reamostragem, gravação do sistema
- **Reconhecimento de Fala:** Sherpa-ONNX offline, VAD, detecção de palavra-chave
- **Protocolos:** WebSocket/MQTT dual, transmissão criptografada, reconexão automática
- **Sistema de Configuração:** Configuração em camadas, acesso por pontos, JSON/YAML

Otimizações de Desempenho

- **Assincronismo Completo:** Sem operações bloqueantes
- **Gerenciamento de Memória:** Cache inteligente e coleta de lixo
- **Áudio de Baixa Latência:** Processamento em 5ms com gerenciamento de fila
- **Controle de Concorrência:** Pool de tarefas, semáforos, segurança de thread

Estrutura do Projeto

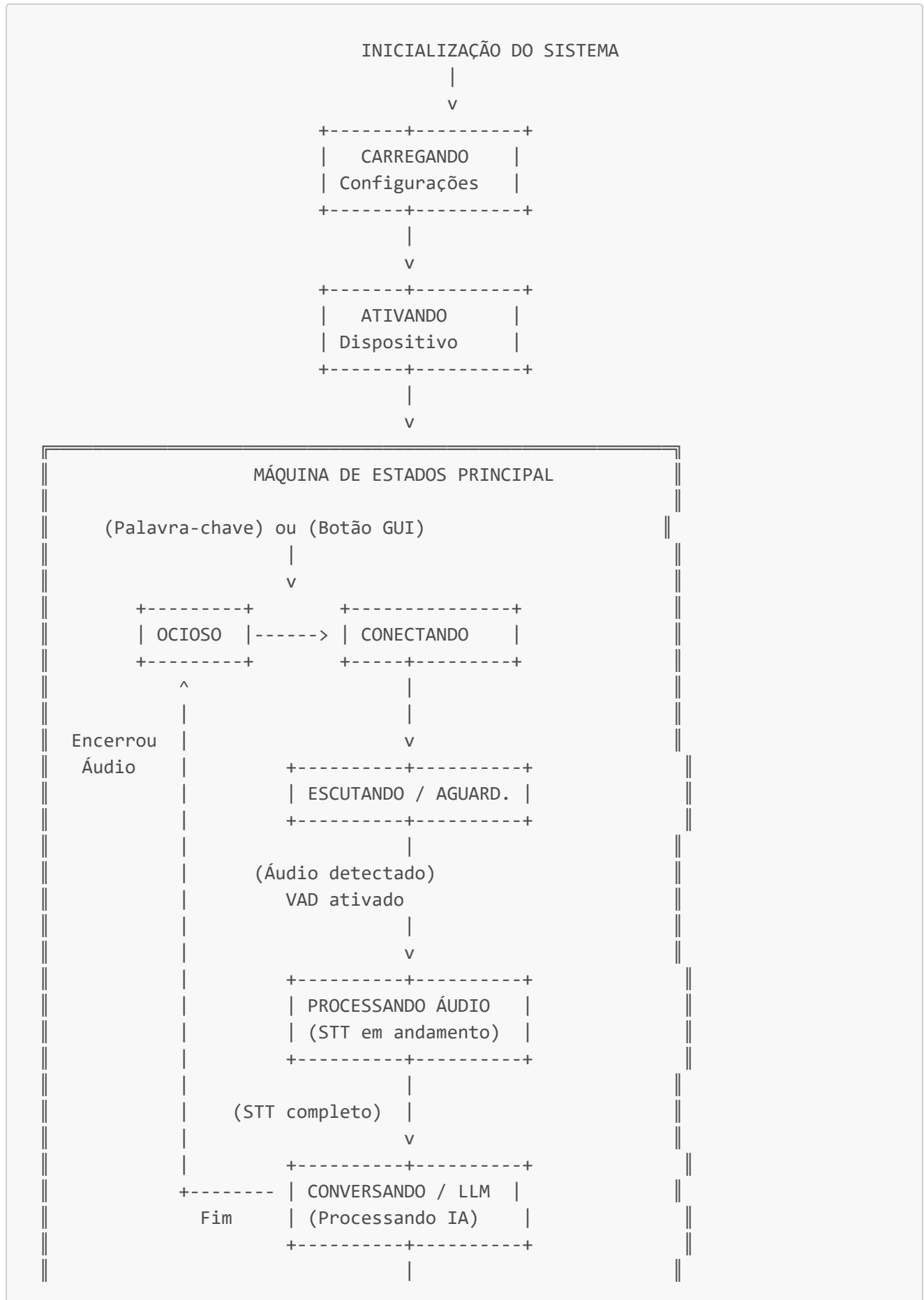
```

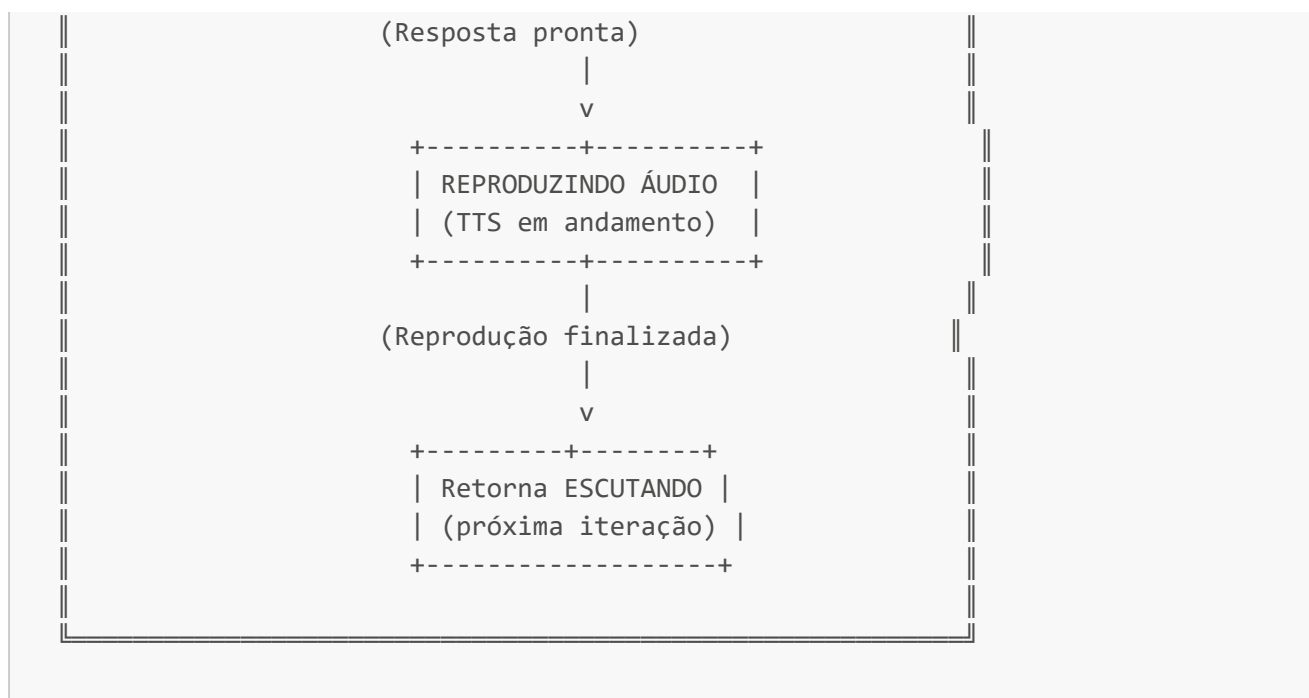
ASSISTENTE-SHI/
├── main.py                # Ponto de entrada
├── src/
│   ├── application.py    # Lógica central
│   ├── audio_codecs/     # Codificação de áudio
│   ├── audio_processing/ # Processamento de áudio
│   ├── core/             # Componentes principais
│   ├── display/          # Camada abstrata de interface
│   ├── iot/              # Gerenciamento IoT
│   ├── mcp/              # Sistema de ferramentas MCP
│   ├── protocols/        # Protocolos de comunicação
│   ├── utils/            # Funções utilitárias
│   └── views/            # Componentes de UI
├── config/               # Arquivos de configuração
├── models/               # Modelos de IA
├── logs/                 # Arquivos de log
└── requirements.txt      # Dependências Python

```

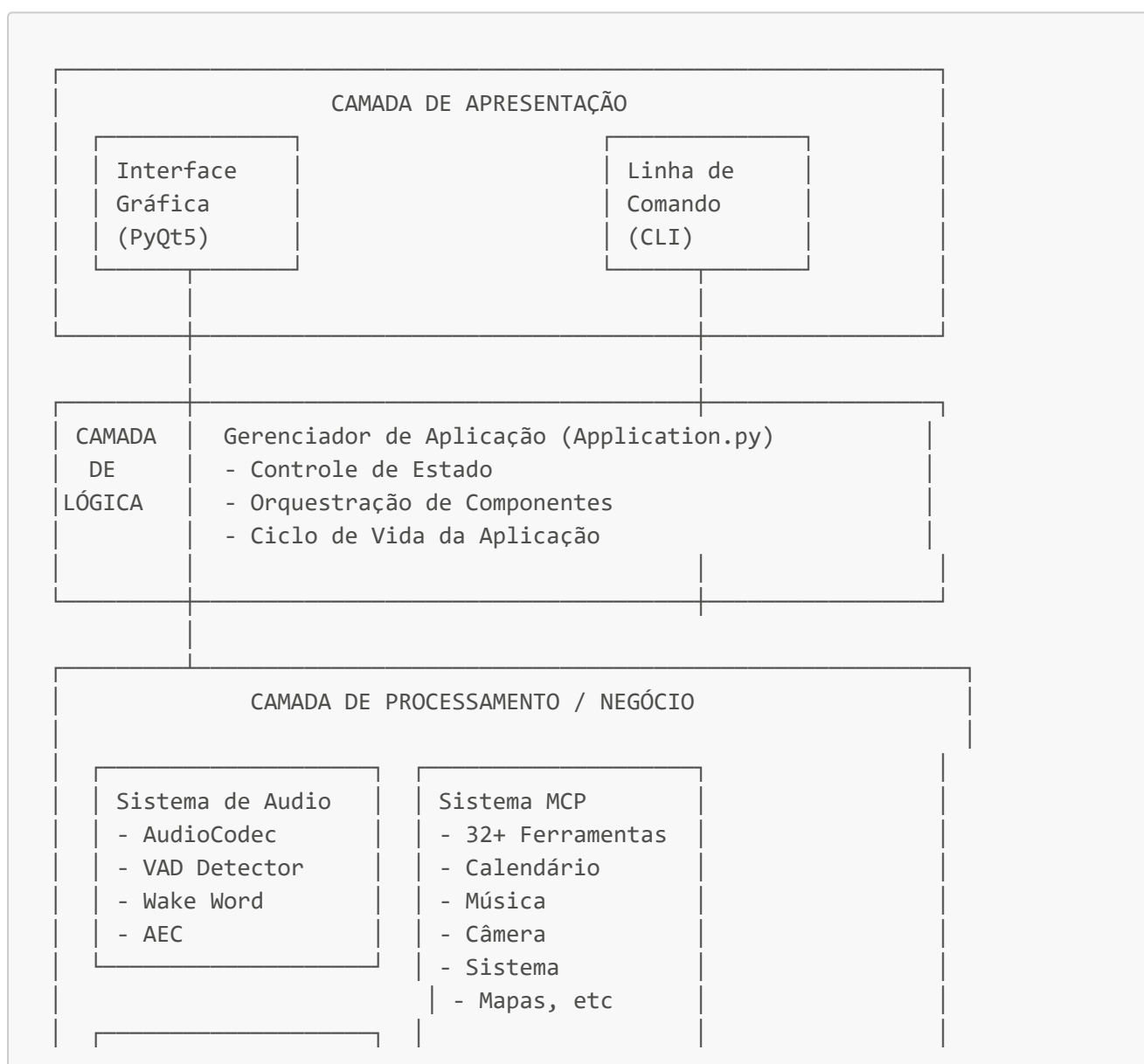
Fluxos e Diagramas do Projeto

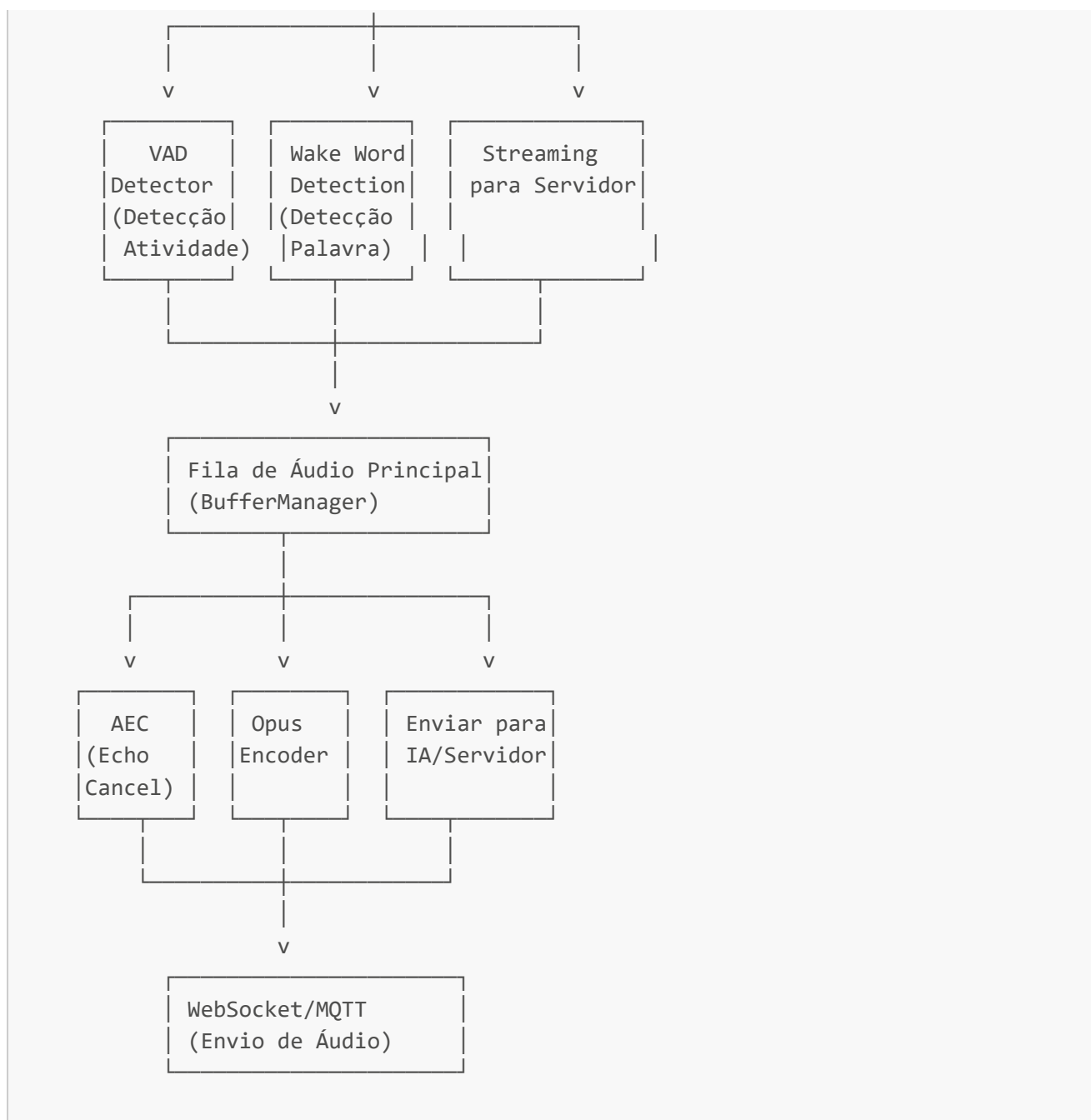
1 Fluxo de Estados da Aplicação



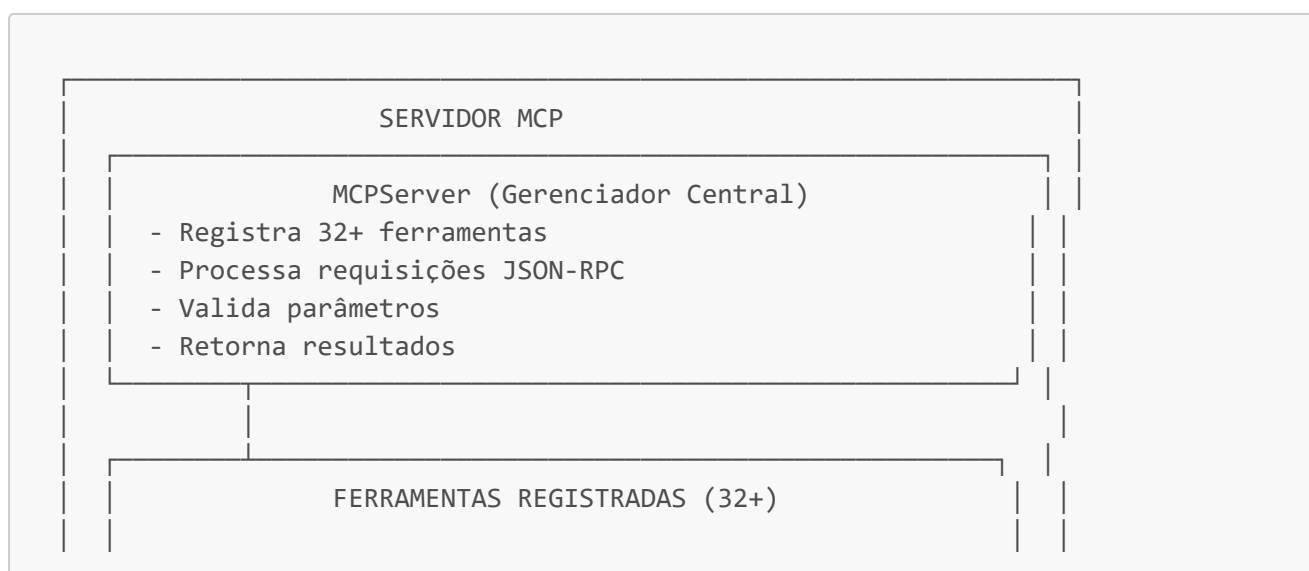


2 Arquitetura em Camadas do Sistema





4 Arquitetura do Sistema MCP (Model Context Protocol)



CONTROLE DO SISTEMA (4 ferramentas)

- set_volume()
- launch()
- get_volume()
- list_running()

GERENCIADOR DE CALENDÁRIO (7 ferramentas)

- create_event()
- update_event()
- get_upcoming_events()
- get_categories()
- get_events()
- delete_event()

REPRODUTOR DE MÚSICA (7 ferramentas)

- search_and_play()
- resume()
- seek()
- get_local_playlist()
- pause()
- stop()
- get_lyrics()

GERENCIADOR DE TEMPORIZADORES (3 ferramentas)

- start_countdown()
- get_active_timers()
- cancel_countdown()

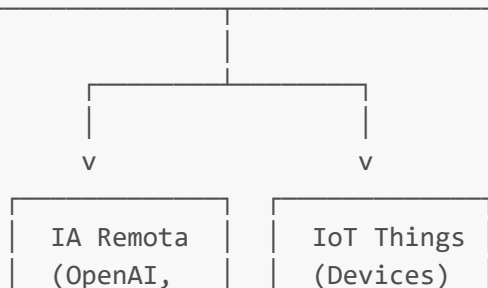
VISÃO COMPUTACIONAL (2 ferramentas)

- take_photo()
- take_screenshot()

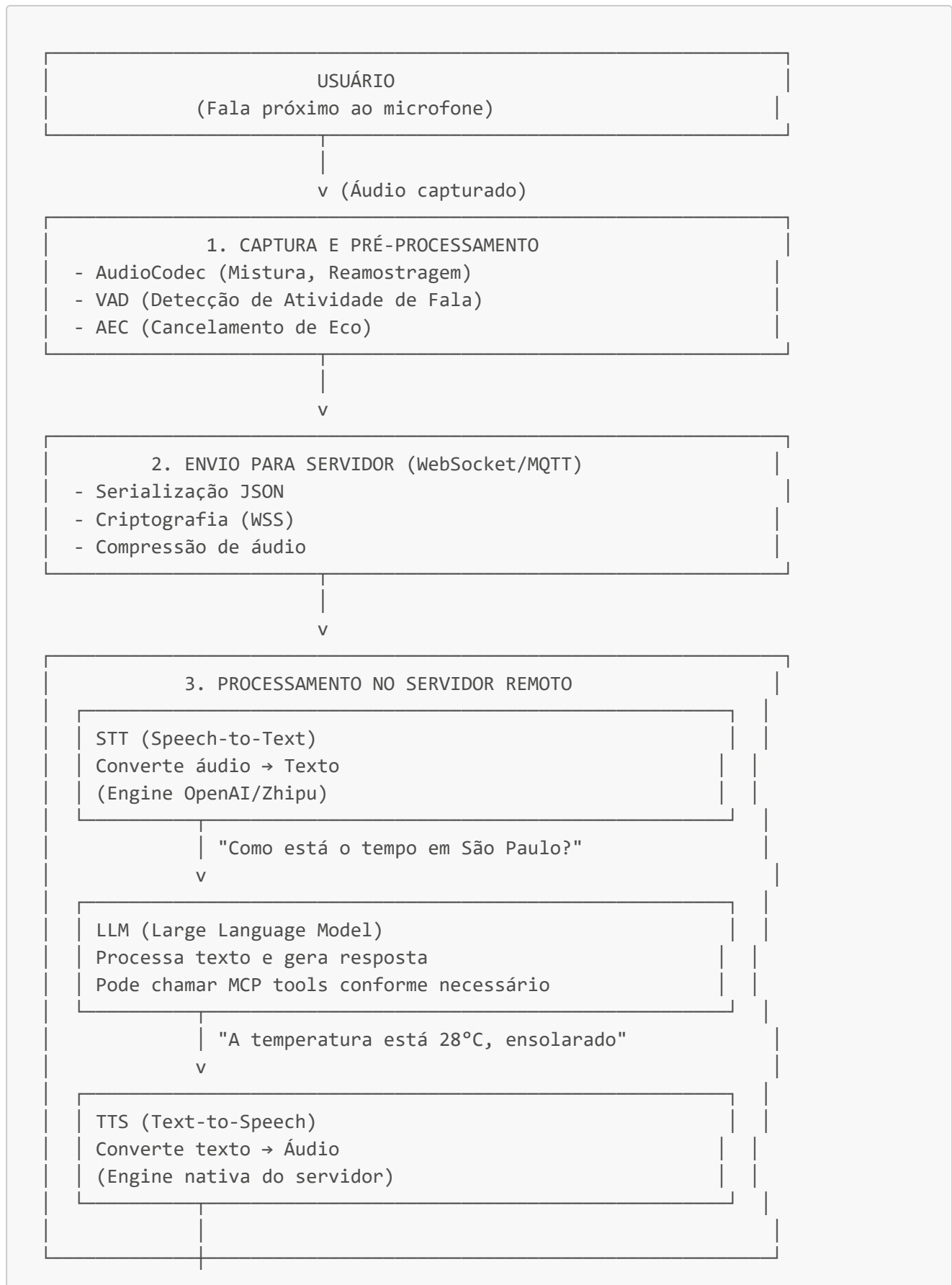
ASTROLOGIA BA ZI (6 ferramentas)

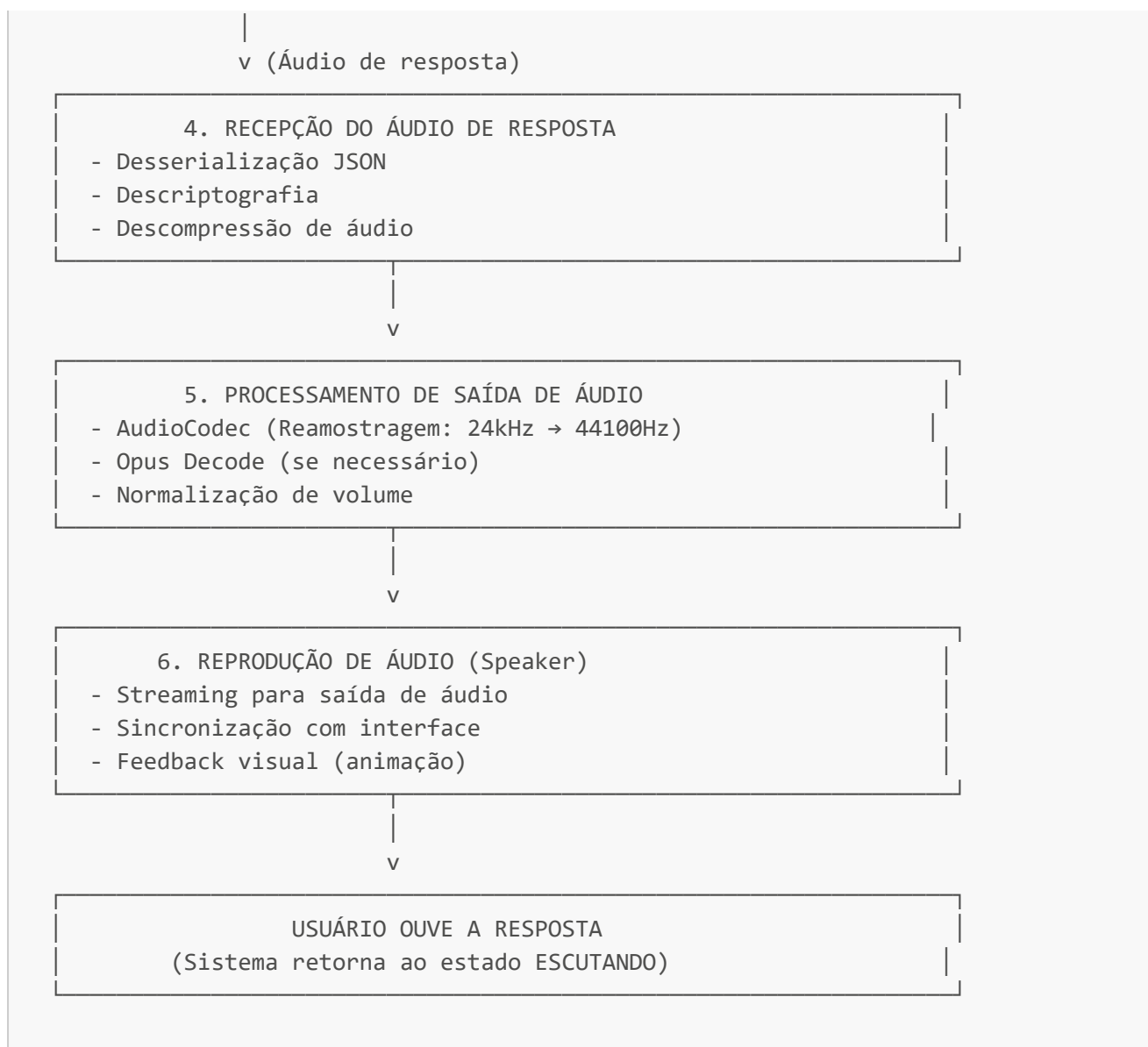
- get_bazi_detail()
- analyze_marriage_compatibility()
- get_chinese_calendar()
- build_bazi_from_lunar_datetime()

+ 3 ferramentas adicionais em desenvolvimento...

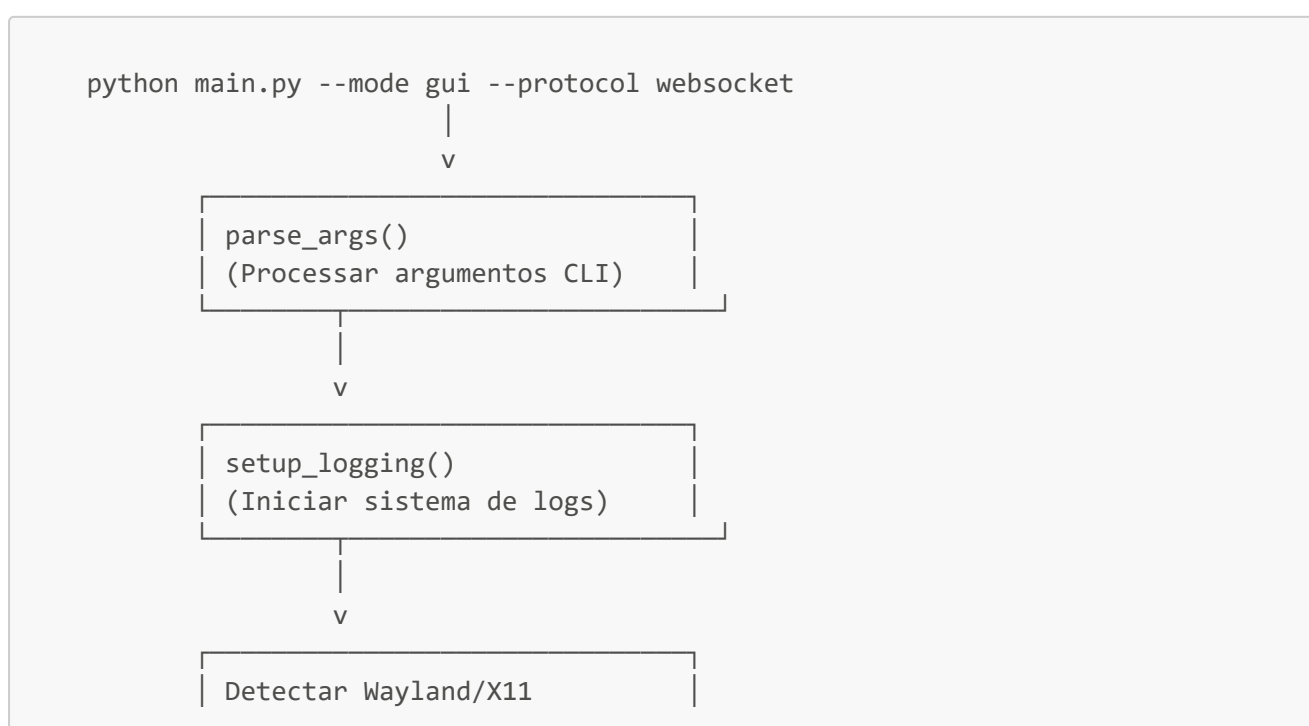


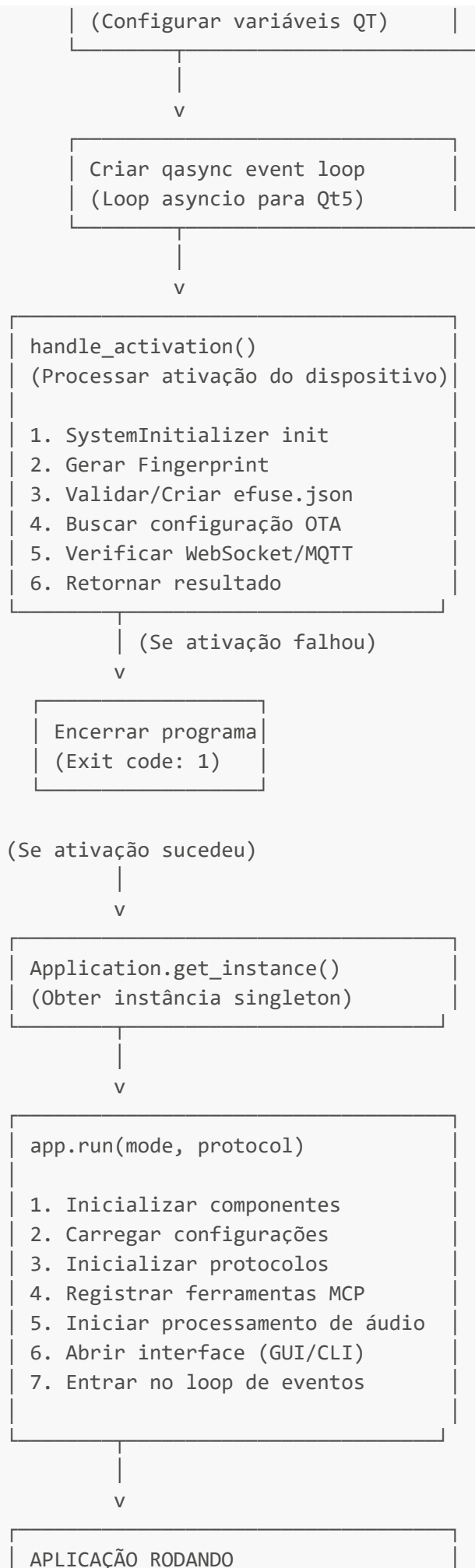
5 Fluxo Completo de Interação Usuário → IA → Resposta

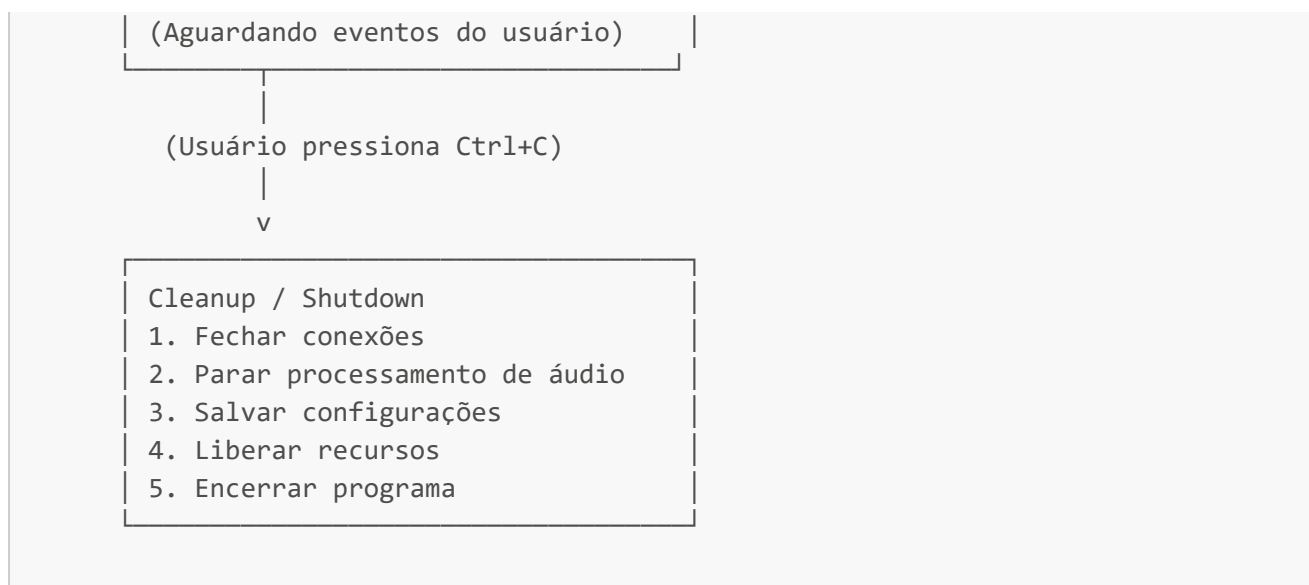




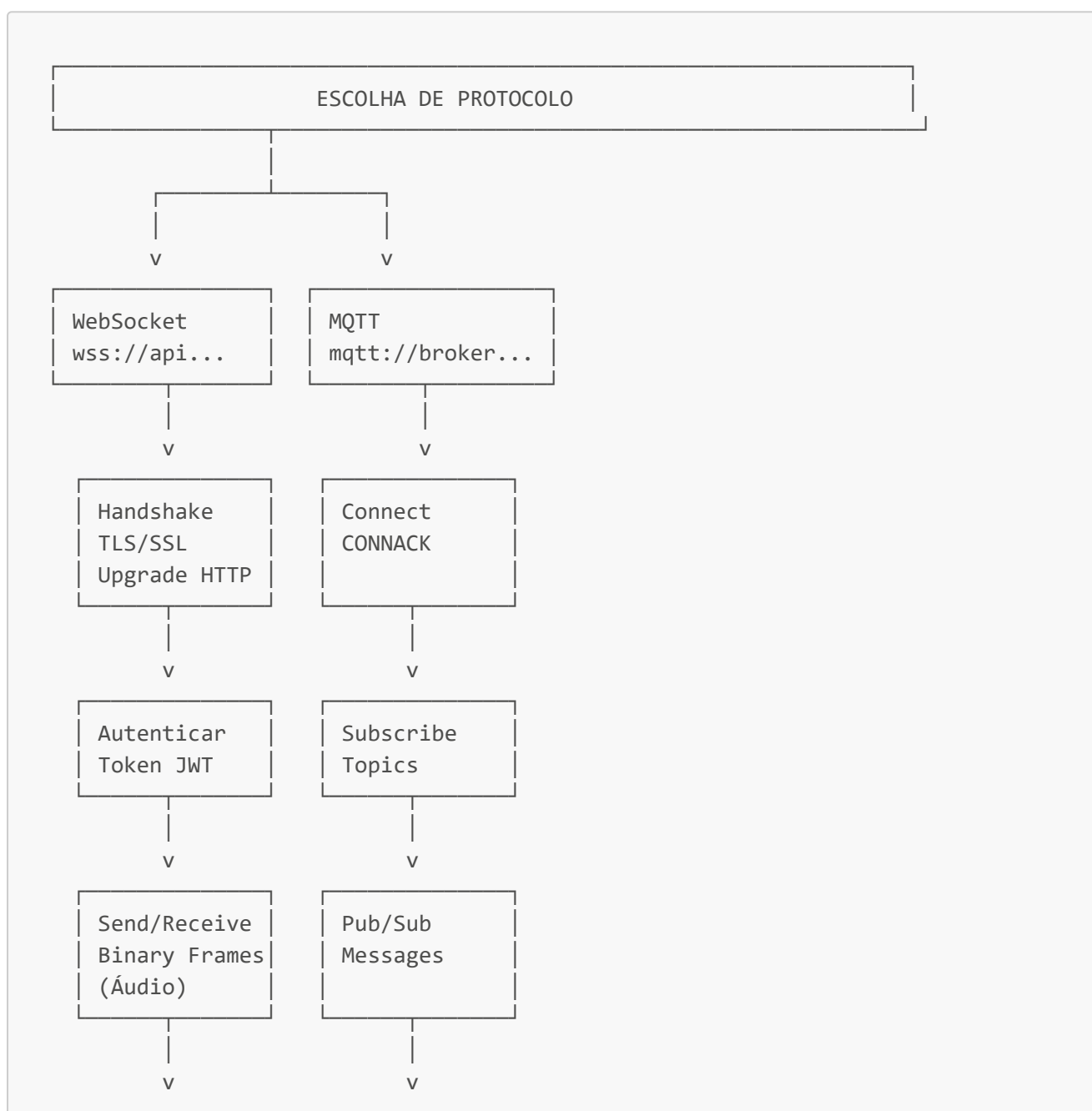
6 Fluxo de Inicialização do Sistema

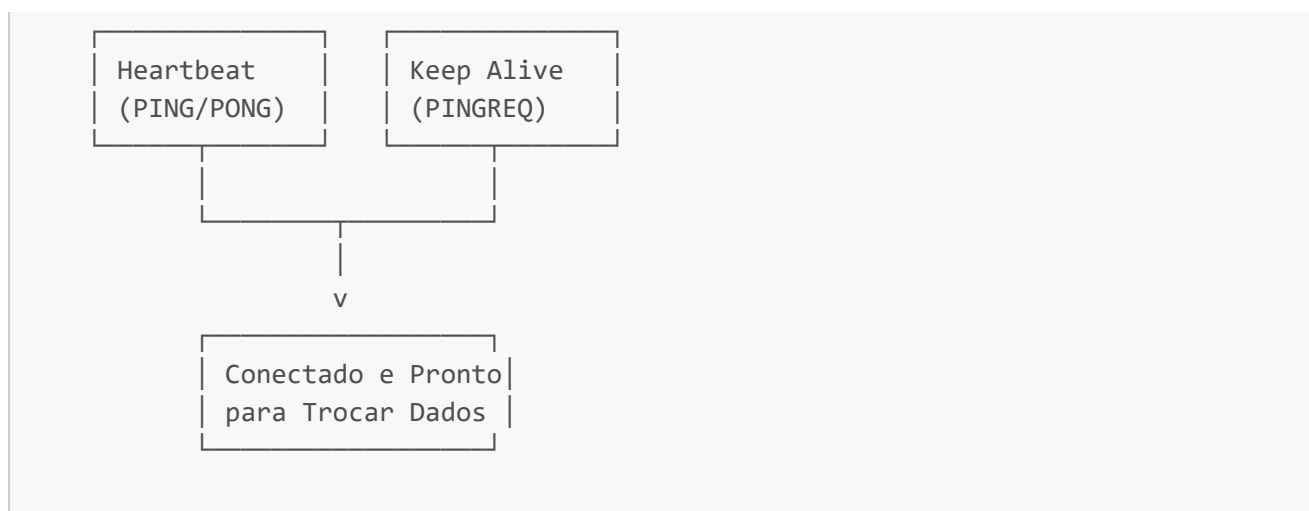




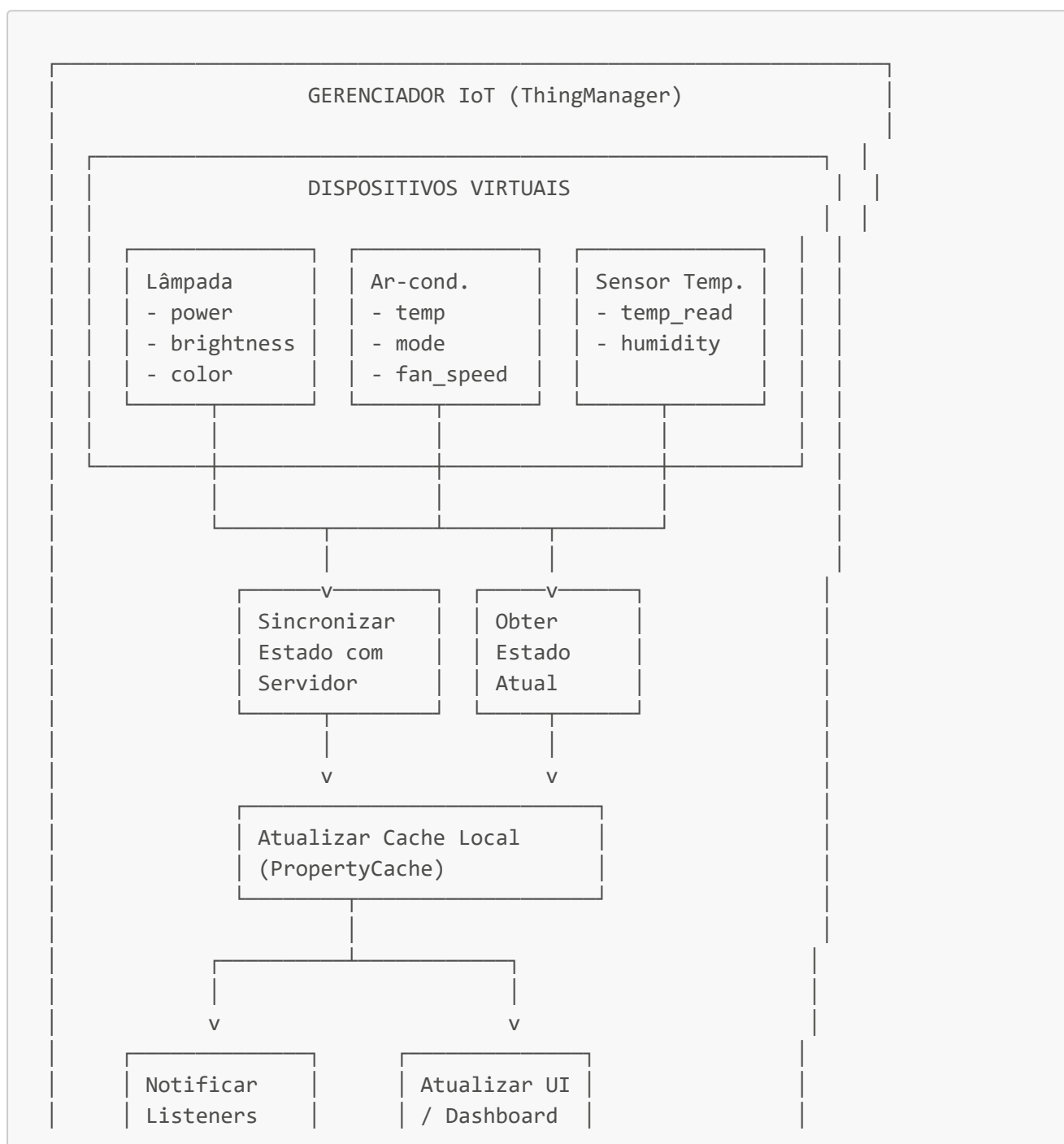


7 Fluxo de Protocolo de Comunicação





8 Fluxo de Sincronização de Estado IoT



Status Atual

☒ Funcional

- ☒ Interface GUI com WebSocket
- ☒ 32 ferramentas MCP registradas
- ☒ Processamento de áudio Opus
- ☒ Interação com IA em tempo real
- ☒ Suporte a câmera (captura de fotos)
- ☒ Gerenciamento de calendário
- ☒ Sistema de lembretes

Conhecido

-  Modelo de Wake Word ausente (Sherpa-ONNX)
-  API de Visão Remota retornando 404
-  Timeout de conexão WebSocket após ~1 minuto em teste

Desenvolvimento

Executar em Diferentes Modos

```
# Modo GUI com WebSocket (padrão)
python main.py --mode gui --protocol websocket

# Modo CLI com MQTT
python main.py --mode cli --protocol mqtt

# Pular processo de ativação (debug)
python main.py --skip-activation
```

Adicionando Novas Ferramentas MCP

Estenda a classe em `src/mcp/tools/` e registre em `mcp_server.py`.

Adicionando Novos Dispositivos IoT

Implemente `Thing` base class em `src/iot/things/`.

Estatísticas do Projeto

- **Ferramentas MCP:** 32+ funções integradas
- **Compatibilidade:** 3 sistemas operacionais

- **Protocolos:** 2 opções (WebSocket, MQTT)
- **Modos:** 2 interfaces (GUI, CLI)

Contribuições

Contribuições são bem-vindas! Por favor:

1. Faça um fork do repositório
2. Crie uma branch para sua feature (`git checkout -b feature/AmazingFeature`)
3. Commit suas mudanças (`git commit -m 'Add some AmazingFeature'`)
4. Push para a branch (`git push origin feature/AmazingFeature`)
5. Abra um Pull Request

Licença

[MIT License](#) - Veja o arquivo LICENSE para detalhes

Agradecimentos

Agradecimentos especiais ao projeto original [py-xiaozhi](#) e todos os contribuidores.

Contato e Suporte

Para dúvidas, sugestões ou relatórios de bugs, abra uma issue no repositório GitHub.

Última atualização: 13 de janeiro de 2026

Status: ☒ Operacional em modo GUI + WebSocket

